Personalized Debiasing for Sequential Recommendation

Ritu Raj Sharma
Dept. Of AI/DS
Intenational Institute of Information Technology Bangalore
Bangalore, India
Ritu.Sharma@iiitb.ac.in

M Vinay
Dept. of CSE
Intenational Institute of Information Technology Bangalore
Bangalore, India
M.Vinay@iiitb.ac.in

Shikhar Mutta
Dept. of CSE
Intenational Institute of Information Technology Bangalore
Bangalore, India
shikhar.mutta@iiitb.ac.in

Abstract*—Sequential recommender systems such as GRU4Rec and SASRec often suffer from popularity bias, where globally popular items dominate recommendations, and temporal trending bias, where recently trending items are over-recommended to all users. Existing debiasing methods such as Inverse Propensity Scoring (IPS) apply the same correction to every user, ignoring differences in user behaviour and their preferences. In this work, we propose a Personalized Debiasing framework that personalizes the debiasing strength for each user based on two identity dimensions: an explorer score α(u) for popularity correction and a trend-follower score β(u) for temporal trending debiasing. We also propose a learned variant where a small MLP jointly trained with the sequential model discovers optimal debiasing parameters end-to-end. These scores are integrated into a personalized exposure-aware loss function without modifying the underlying sequential architecture. Experiments on the MovieLens-1M dataset demonstrated that the proposed method achieves better recommendation diversity and exposure fairness while maintaining the ranking quality. Compared to standard IPS debiasing, our method achieves better Recall@10, higher coverage, and improved long-tail exposure, validating the effectiveness of personalized debiasing over uniform correction strategies. The learned MLP variant further improves ranking quality, achieving approximately 3–4% higher Recall@10 than the fixed personalized model, while maintaining similar fairness and diversity metrics. Overall, the results demonstrate that personalized debiasing is more effective than uniform correction for balancing recommendation quality, fairness, and diversity.

Keywords— Sequential Recommendation, Popularity Bias, Temporal Trending Bias, Inverse Propensity Scoring, User Identity, Personalized Debiasing

I. INTRODUCTION

Recommender systems have become an essential part of almost every online platform today, from Netflix and Spotify to Amazon and YouTube. These systems try to predict what a user might want to see, listen to, or buy next, based on their past behaviour. Among the many approaches used for this, sequential recommendation has gained a lot of attention in recent years. In sequential recommendation, the system looks at the order in which a user has interacted with items (such as movies watched or products viewed) and tries to predict the next item in that sequence. Models like GRU4Rec [7] and SASRec [8] use recurrent or attention-based neural networks to capture these sequential patterns.

While these models have shown good results in terms of accuracy, they are known to suffer from certain biases that come from the training data itself. Two of the most prominent biases are popularity bias and temporal trending bias.

Popularity bias is the tendency of recommendation models to over-recommend items that are globally popular, regardless of individual user preferences. This happens because popular items appear very frequently in the training data, and the model learns to associate them with high relevance for almost every user. Abdollahpouri et al.[1] studied this problem from the user's perspective and found that different users are affected differently. They defined three groups of users: Niche (users who prefer non-popular items), Diverse (users with mixed preferences), and Blockbuster-focused (users who mainly like popular items). Their key finding was that Niche users suffer the most from popularity bias even though more than 50% of their profile consists of non-popular items. Almost all their recommendations are popular items. This shows that popularity bias is fundamentally an unfairness problem that affects different users differently.

Temporal trending bias is a separate but equally important issue. This occurs when items that are currently trending across the platform get over-recommended to all users, regardless of whether a particular user follows trends or has stable, long-term interests. Zhang et al. [5] showed that item popularity is not static. It changes across time stages. An item that was popular last month may not be popular today, and vice versa. When models are trained on data where recent interactions are dominated by a few trending items, they learn to push these trending items to everyone. This is problematic because not every user is a trend-follower. Some users consistently prefer classic, stable content over whatever is trending at the moment.

One of the approaches to handle such biases is Inverse Propensity Scoring (IPS), introduced by Schnabel et al. [2] for recommendation settings. IPS reweights each training example by the inverse of its exposure probability, so that popular items get down weighted and rare items get amplified. However, IPS applies the same correction to every user. A Niche user and a Blockbuster-focused user both receive the same debiasing strength, which is clearly suboptimal by the findings of Abdollahpouri et al. [1].

More recently, Ning et al. [3] proposed the idea of Personal Popularity (PP), which computes a separate popularity score for each user-item pair rather than using a single global score. Their PPAC framework showed impressive improvements, demonstrating that personalized debiasing significantly outperforms uniform approaches. However, their work is designed for general collaborative filtering models (MF, LightGCN) and does not address sequential recommendation or temporal trending bias.

Yang et al. [4] brought debiasing into the sequential domain with their DRO-based framework. They showed that exposure bias in sequential models leads to learned user preferences that do not align with true interests, and proposed a distributionally robust optimization approach that works with both GRU4Rec and SASRec. While effective, their method also applies uniform correction to all users without distinguishing between user types.

In this work, we propose a Personalized Debiasing framework that addresses these gaps. Our key contributions are as follows:

- **Personalized Dual-Bias Correction:** We introduce a loss function that jointly corrects popularity bias and temporal trending bias with separate, user-specific debiasing strengths, controlled by an explorer score $\alpha(u)$ and a trend-follower score $\beta(u)$.
- **User Identity Modelling:** We propose interpretable user identity scores computed using category entropy and temporal trending correlation to capture differences in user preference diversity and trend-following behavior.
- **Model-Agnostic Design:** Our framework operates at the loss function level and can be applied to any sequential model (GRU4Rec, SASRec) without architectural changes. The debiasing is absorbed during training, requiring zero extra computation at inference.

II. Background And Related Works

A. Popularity Bias and Its Unfairness Across Users

Popularity bias occurs when recommendation algorithms over-recommend items that have high global interaction counts, at the expense of less popular (long-tail) items. This happens primarily because training data follows a long-tail distribution (a small number of items receive the majority of interactions), while the vast majority of items are rarely interacted with. When models are trained on such skewed data, they learn that recommending popular items is a safe strategy that leads to low training loss, even if it means ignoring the specific preferences of individual users.

Abdollahpouri et al. [1] studied this problem from the user's perspective using the MovieLens 1M dataset. They divided users into three groups based on the ratio of popular items in their profiles: Niche (bottom 20%, more than half their profile is non-popular items), Diverse (middle 60%), and Blockbuster-focused (top 20%, more than 85% popular items). A key finding was the strong negative correlation between profile size and popular item ratio. That means the most active users who contribute the most data are also the ones most interested in non-popular items.

They proposed the Group Average Popularity (GAP) metric and its relative change ΔGAP to quantify how much algorithms push popularity beyond what users expect. Their results showed that Niche users experience the largest ΔGAP across all tested algorithms (User KNN, Item KNN, SVD++, Biased MF), meaning the gap between what they want and what they receive is the most severe. This finding is central to our work. It directly motivates why debiasing should be user-specific rather than uniform.

B. Temporal Dyamics of Item Popularity

A critical insight that extends beyond static popularity bias is that item popularity changes over time. Zhang et al. [5] addressed this through their PDA (Popularity-bias Deconfounding and Adjusting) framework. They modelled item popularity as a confounder in a causal graph. Popularity influences both, which items get exposed to users and which items users interact with, creating a spurious correlation that does not reflect true user preference.

Their key contribution relevant to our work is the concept of local popularity, tem popularity computed within a specific time stage rather than globally across the entire dataset. They showed that the same item can be highly popular in one time period and unpopular in another. This dynamic view of popularity has two implications for our framework. First, it provides the foundation for our trending exposure computation e_trend(i, t), where we measure how popular an item is within a recent time window rather than using a static global count. Second, it motivates the idea that temporal trending is a separate bias dimension from static global popularity. An item can be globally popular but not currently trending (e.g., a classic film), or currently trending but not globally popular (e.g., a newly released indie film).

This concept of local temporal popularity has also been recently validated in the sequential recommendation setting by TALE [6], which uses trend-aware normalization based on item popularity within a recent window to prevent fad items (items that are popular only during specific periods) from being treated the same as steady sellers (items that have maintained decent attraction over a long period).

C. Debiasing Approaches for Recommendation

Schnabel et al. [2] introduced the foundational framework for debiasing in recommendation by treating recommendations as treatments and using Inverse Propensity Scoring (IPS). In their framework, each training example is reweighted by the inverse of its propensity (exposure probability). Items with high exposure get down weighted, and items with low exposure get amplified. This ensures that the model does not overfit to frequently exposed items. While simple and elegant, this approach applies the same correction to all users without considering individual differences.

Yang et al. [4] uses debiasing specifically into the sequential recommendation domain. They proposed a Distributionally Robust Optimization (DRO) framework that optimizes the worst-case error over an uncertainty set constructed from system exposure data. Their method introduces a penalty for items with high exposure probability during training. They demonstrated their approach on both GRU4Rec and SASRec backbones, showing it is model-agnostic. Importantly, they showed that standard IPS methods suffer from high variance in the sequential setting, and DRO provides a more robust alternative. However, like standard IPS, their DRO framework does not differentiate between users. It applies the same robust optimization to all users regardless of their individual relationship with popularity or trending content.

D. Personalized Debiaisng

The most closely related work to ours is the PPAC framework by Ning et al. [3]. They identified the fundamental limitation of global popularity i.e. it uses a single set of popular items for all users and therefore cannot capture individual user interests. They proposed Personal Popularity (PP), which computes item popularity among similar users (using Jaccard similarity on interaction sets) rather than across all users globally. Their causal framework uses separate MLPs to predict PP and GP values, and applies counterfactual inference at inference time to adjust the direct effects of both popularity types on the prediction score.

Their results showed remarkable improvements, with gains of up to 46.8% in Recall and 61.9% in NDCG over existing baselines on three datasets (MovieLens-1M, Gowalla, Yelp2018). Their ablation study confirmed that removing PP hurts more than removing GP in most cases, validating that personalization is the key ingredient.

While PPAC represents a major advancement, it has several limitations that our work addresses. First, PPAC operates on general collaborative filtering models (MF, NCF, LightGCN) and has not been applied to sequential recommendation where the temporal ordering of interactions creates additional bias dynamics. Second, it only addresses popularity bias and does not consider temporal trending bias as a separate dimension. Third, the personalization is at the user-item level (each user-item pair gets a different PP score), which requires computing pairwise Jaccard similarity for all users. We argue that a simpler user-level identity score is sufficient and more computationally efficient. Fourth, PPAC's debiasing operates at inference time through counterfactual score adjustment, requiring extra models at inference. Our debiasing operates at training time through loss reweighting, meaning the deployed model has zero extra overhead.

E. Research Gap

To summarize the gaps identified from our review of existing literature: no existing work simultaneously provides

1) Personalized debiasing where the correction strength is adapted per user identity
2) Dual-bias correction for both static popularity and dynamic temporal trending,
3) Applied specifically to the sequential recommendation setting, and
4) With both formula-based and learnable identity parameters.

Additionally, while within-sequence recency bias (the model over-emphasizing the last few items in a sequence) has been measured [9], the platform-level temporal trending bias we address is a distinct and complementary problem. Our proposed framework fills these gaps.

III. Proposed Methodology

A. Problem Formulation

We consider the standard next-item prediction setting. Given a user u with an interaction sequence $[i_1, i_2, \dots, i_t]$ ordered by time, the objective is to predict the next item i_{t+1} that the user will interact with. A sequential model (such as GRU4Rec [7]) processes this sequence and outputs a probability distribution $P(i, t+1 \mid i_1, i_2, \dots, i_t)$ over all candidate items. The standard training loss is the cross-entropy loss:

$$L_{standard} = -\log P(i_{t+1})$$

Our goal is to modify this loss function to correct for popularity bias and temporal trending bias in a user-specific manner, without changing the model architecture itself.

B. Exposure Estimation

Popularity Exposure: For each item i, we compute its popularity exposure $e_{pop}(i)$ as the normalized interaction frequency in the training data:

$$e_{pop}(i) = \frac{freq(i)}{\sum_i freq(i)}$$

where, $freq(i)$ is the number of times item *i* appears in the training data across all users and all time periods. If the training data has 1,000,000 total interactions, a blockbuster movie with 80,000 interactions gets e_{pop} = 80,000 / 1,000,000 = 0.08. A niche indie film with 2,000 interactions gets e_{pop} = 2,000 / 1,000,000 = 0.002. The blockbuster captured 8% of all platform interactions while the indie film captured only 0.2%.

Trending Exposure. Unlike popularity exposure which is a static global property, trending exposure captures how popular an item is at a specific point in time. Following the local temporal popularity concept from Zhang et al. [5], we compute trending exposure within a sliding time window. For each item *i* at time *t*:

$$e_{trend}(i, t) = \frac{count(i, [t-W, t])}{\sum_i count(i, [t-W, t])}$$

where $count(i, [t-W, t])$ is the number of interactions with item *i* in the window of *W* days before time *t*, and *W* is a hyperparameter (we use *W* = 30 days as default). This captures how trending an item is at a specific moment. An item with a high e_{trend} value (e.g., 0.05) captured 5% of all platform interactions during that window, indicating it was heavily trending. An item with e_{trend} close to 0 (e.g., 0.0002) received a negligible share of interactions, meaning it was barely being noticed during that period.

The key distinction between e_{pop} and e_{trend} is that e_{pop} captures all-time popularity (a classic that has been popular for years) while e_{trend} captures current momentum (a new release that is trending right now). An item can have high e_{pop} but low e_{trend} (a classic film that was popular overall but is not trending this week), or low e_{pop} but high e_{trend} (a newly released film that is currently trending but has limited total interactions). This separation allows our framework to correct for both biases independently.

For computational efficiency, we discretize time into bins (e.g., weekly) and precompute interaction counts for each item within each bin rather than recomputing sliding windows for every interaction. This design aligns with recent sequential recommendation approaches such as TALE [6], which also leverage temporally localized popularity statistics for trend-aware normalization. Both e_{pop} and e_{trend} are precomputed before training and remain fixed throughout.

C. User Identity Modeling

The central idea of our work is that different users need different levels of debiasing. We capture this through two aspects of user behavior, each controlling the correction for one type of bias.

Explorer Score α(u) (Controls Popularity Debiasing): This measures how diverse a user's taste is across item categories. A user who interacts with items across many different genres (an explorer) needs strong popularity debiasing. They genuinely benefit from discovering niche items that standard models would suppress. A user focused on one or two genres (a loyal user) may actually prefer popular items within their favorite categories and needs minimal correction.

This behavioral aspect is directly motivated by Abdollahpouri et al.'s [1] finding that users differ in their popularity propensity. Their discrete grouping (Niche, Diverse, Blockbuster-focused) based on the ratio of popular items in user profiles captures this intuition qualitatively. We formalize it as a continuous score using normalized Shannon entropy over the user's category distribution:

$$H(u) = -\sum c\, P(c|u) \times log\, P(c|u)$$

$$\alpha(u) = \frac{H(u)}{log(K)}$$

where $P(c|u)$ is the fraction of user u's interactions that belong to category c, and K is the total number of categories (e.g., 18 genres in MovieLens). The normalization by log(K) ensures α falls in [0, 1] with a principled information-theoretic interpretation: α = 1 means perfectly uniform distribution across all categories (maximum exploration), and α = 0 means all interactions in a single category (complete loyalty). The use of entropy to capture user diversity propensity in recommendation has been previously studied by Wang et al. [9], who used Shannon entropy over genre or categories of items distributions to personalize diversity in re-ranking.

For example, consider two users. User1 has watched movies spread equally across Horror, Thriller, Drama, Sci-Fi, and Romance. User2 has watched 9 Action movies and 1 Comedy. So, User1 entropy will be higher than User2.

Trend-Follower Score β(u) (Controls Trending Bias Debiasing): This measures how much a user tends to interact with items that are currently trending on the platform. A trend-follower who mostly interacts with currently popular items needs less trending correction, because trending items are genuinely relevant to them. A stable-interest user who watches classic or older content needs strong trending correction to avoid being overwhelmed by whatever is trending this week.

We compute β(u) by averaging the trending exposure scores across all items in the user's interaction history:

$$\beta(u) = \left(\frac{1}{|history_u|}\right) \times \sum_{(i,t)\in history_u} e_{trend}(i,t)$$

This computes the average "trendiness" of items a user interacts with, measured at the time of each interaction. The value falls naturally between 0 and 1 without additional normalization. A user who mainly watches newly released trending content at the time of interaction gets high β (e.g., 0.8), while a user who watches classic films or stable long-term favourites gets low β (e.g., 0.1). This approach uses the same local temporal popularity concept from PDA [5] at the user level. Instead of computing trending exposure for items, we aggregate it per user to create a user identity score.

Both α and β are precomputed once from the training data and stored as a lookup table, adding zero overhead during training.

D. Identity-Aware Loss Function

We now combine the exposure estimates and identity scores into a single debiased training loss. The core idea is to reweight each training example by the inverse of its exposure, but raise the exposure to a user-specific power so that the correction strength is personalized.

$$\mathrm{L} = \left(\frac{1}{e_{pop}(i_{t+1})^{\alpha(u)} \times e_{trend}(i_{t+1})^{1-\beta(u)}}\right) \times (-log\, P(i_{t+1}))$$

Let us understand how each component works:

Popularity correction via $e_{pop}(i)^{\alpha(u)}$: When α(u) is large (explorer users), the model strongly amplifies learning signals from rare items because $e_{pop}$is small for such items. When α(u) is small (loyal users), the correction is weak, allowing popular items to dominate learning signals.

Trending correction via $e_{trend}(i)^{1-\beta(u)}$: For stable-interest users (low β), the exponent 1- β is large, which increases the learning weight of non-trending items and reduces the dominance of trending items. For trend-following users (high β), the exponent becomes small and the correction diminishes, allowing trending items to remain influential.

E. Learned Identity Variant

The formula-based α and β are simple and interpretable, but they may not capture all the nuances of user behaviour. For instance, a user with high entropy computed from only 5 interactions is unreliable. The formula cannot distinguish this from a user with high entropy from 500 interactions. A user might have medium entropy but very low average popularity, suggesting they are more of an explorer than entropy alone indicates. To address these limitations, we propose a learned variant where a small Multi-Layer Perceptron (MLP) produces α and β from multiple user features.

The MLP takes four input features for each user: (1) entropy (the same normalized Shannon entropy used in the

formula-based) α, (2) average popularity (the mean popularity of items in the user's history, measuring how mainstream their taste is), (3) profile size (the number of interactions (normalized), indicating how reliable the other features are), and (4) trending correlation (the same trend-follower score used as formula-based) β.

$$hidden = ReLU(W_1 \cdot x(u) + b_1)$$

$$[\alpha(u), \beta(u)] = sigmoid(W_2 \cdot hidden + b_2)$$

This is a single MLP with two output neurons: one for α and one for β. We use a single shared MLP rather than two separate networks because both outputs depend on the same input features and are conceptually related (both capture aspects of user identity). The sigmoid activation ensures both values remain in (0, 1).

Critically, this MLP trains jointly with the sequential model through a single optimizer. Unlike the PPAC framework [3], which trains its PP and GP estimation MLPs with direct supervised signals (MSE loss against observed popularity values), our User Profile MLP has no direct supervision for α and β. Instead, it learns what debiasing strength minimizes the overall recommendation loss. The gradient flows backward from the recommendation loss, through the weight computation, into the MLP parameters. Over training, the MLP converges to producing debiasing strengths that are optimal for each type of user.

At inference time, the User Profile MLP is not used. The debiasing has already been absorbed into the sequential model's learned weights during training. The model has learned better embeddings for rare and non-trending items because those items received amplified loss during training. This means zero additional inference cost, unlike PPAC [3] which requires running three models (base + PP MLP + GP MLP) plus counterfactual score adjustment at inference time.

F. Architecture Overview

The complete system consists of two neural components trained jointly: (1) the sequential recommendation backbone (GRU4Rec or SASRec), which processes item sequences and predicts the next item, and (2) the User Profile MLP (only in the learned variant), which takes precomputed user features and outputs α and β.

The training pipeline for each batch is: the sequential model processes the item sequence and outputs the prediction probability $P(i_{t+1})$ the User Profile MLP takes the user's feature vector and outputs α(u) and β(u); the weight is computed from precomputed e_{pop} *and* e_{trend} values using the user behaviour parameters; the weighted loss is computed and backpropagated through both networks simultaneously via a single optimizer. The exposure values and user features are all precomputed before training as a one-time preprocessing step.

For the formula-based variant, the User Profile MLP is replaced by a simple lookup of precomputed α and β values, making the system even simpler. Only the standard sequential model is trained, with modified loss weights.

IV. Experimental Design

A. Datasets

We have evaluated our method on MovieLens-1M datasets. MovieLens-1M contains approximately 1 million ratings from 6,000 users on 4,000 movies, with timestamps and genre metadata (18 genres). We convert to implicit feedback by keeping ratings ≥ 4 as positive interactions.

For data splitting, we follow the per-user temporal holdout strategy: for each user, the last interaction forms the test set, the second-last forms the validation set, and the rest forms the training set. This ensures strict temporal ordering with no future data leakage.

All our experiments were conducted using the following preprocessing configuration:

```
data:
   data_path: data/ml-1m
   min_rating: 4.0
   min_interactions: 5
```

Ratings that are ≥ 4 were treated as positive feedback, users with less than 5 interactions were removed

B. Baselines

We compare the following methods:

Table 1: Comparison of Experimental Models

Model	Description
Popularity	Recommends most globally popular items to all users. Non-personalized baseline.
GRU4Rec	Standard GRU-based sequential model with cross-entropy loss. No debiasing.
GRU4Rec + IPS	GRU4Rec with standard IPS reweighting (Schnabel et al. [2]). Same correction for all users.
Ours (Formula)	GRU4Rec + Personalized debiasing with formula-based α and β.
Ours (Learned)	GRU4Rec + Personalized debiasing with MLP-learned α and β.

C. Common GRU4Rec Model Configuration

The following configuration was used across all experiments that uses GRU4Rec as the underlying sequential model:

model:

```
embed_dim: 128
      hidden_dim: 256
      num_layers: 1
      dropout: 0.4
      max_seq_len: 50
```

The model uses 128 dimensional embeddings and a GRU hidden state of 256

D. Experiment Environment and Hardware Setup

All the experiments were conducted using PyTorch library, training and evaluation were conducted on a CUDA enabled

GPU environment to accelerate sequential model training and large-batch recommendation evaluation.

GPU acceleration was enabled through:

```
training:
  device: cuda
```

All tensors and model computations execute on GPU rather than CPU, significantly reducing training time.

To improve computational efficiency, Automatic Mixed Precision (AMP) was enabled. AMP performs selected computations using lower precision floating point operations while maintaining numerical stability. This improves throughput and reduces GPU memory usage.

```
use_amp: true
```

Data was loaded using parallel workers. Using multiple workers allows data preprocessing and batch preparation to occur in parallel with GPU computation. Pinned memory accelerates CPU-to-GPU memory transfer.

```
num_workers: 4
pin_memory: true
```

The experiments were trained using mini-batch optimization on the GPU with a batch size of 512. Early stopping based on validation performance was used to prevent overfitting.

The following steps are performed in the training pipeline:

1. Dataset preprocessing and sequence generation
2. Computation of popularity exposure statistics
3. Computation of temporal trending exposure statistics
4. Computation of user identity scores $\alpha(u)$ and $\beta(u)$
5. Mini-batch GRU4Rec training on GPU
6. Validation-based checkpoint selection
7. Final evaluation on the test set

For each training iteration following steps were performed:

- user interaction sequences were converted into fixed-length padded sequences,
- embeddings were generated for each item,
- the GRU encoder processed the sequence,
- personalized exposure-aware weights were computed,
- weighted cross-entropy loss was calculated,
- gradients were backpropagated through the network,
- and optimizer updates were performed using Adam.

The personalized exposure weights were clipped using:

```
max_weight: 5.0
```

This prevents extremely rare items from generating excessively large gradients during training to stabilize optimization.

The use of GPU acceleration, AMP, and parallel data loading enabled efficient experimentation across multiple debiasing configurations and temporal window settings.

E. Common Training Configuration

The following training configuration was shared across all experiments:

```
training:
  batch_size: 512
  lr: 0.001
  weight_decay: 0.0001
  epochs: 50
  patience: 10
  eval_k: 10
  device: cuda
  num_workers: 4
  pin_memory: true
  use_amp: true
debiasing:
  loss_type: ce
  trend_window_days: 30
  max_weight: 5.0
  use_alpha: true
  use_beta: true
```

All models were trained using the Adam optimizer with a learning rate of 0.001 and weight decay of 0.0001. Training was performed for a maximum of 50 epochs with early stopping (patience = 10) to prevent overfitting. Evaluation was performed using Top-K metrics with K = 10.

F. Configuration for Main Comparison

1) Popularity

```
loss_type: ce
```

The popularity baseline recommends items purely based on global interaction frequency without sequential modeling or personalization. It is included to compare against learned recommendation approaches.

2) GRU4Rec CE Baseline

```
loss_type: ce
```

This baseline trains GRU4Rec using standard cross-entropy loss without exposure-aware weighting. It serves as the standard sequential recommendation baseline.

3) GRU4Rec IPS Baseline

```
loss_type: ips
```

This configuration applies inverse propensity scoring (IPS) for uniform exposure debiasing across all users it is not personalized according to user behavior.

4) GRU4Rec Personalized (Formula Based)

```
loss_type: personalized
```

This is the proposed personalized dual-bias debiasing framework. It jointly uses $\alpha(u)$ for popularity debiasing and $\beta(u)$ for temporal trend debiasing, enabling adaptive exposure correction based on individual user behavior.

5) GRU4Rec Learned (MLP)

```
loss_type: personalized_learned
```

In addition to the fixed formula based formulation, we propose a learned debiasing model where a lightweight MLP is jointly trained with GRU4Rec to estimate personalized exposure weights α(u), β(u) learned via MLP, providing end-to-end optimization.

G. Configuration for Ablation Studies

a) Component Ablation

1) Alpha-only Configuration

```
use_alpha: true
use_beta: false
```

The configuration just enables only popularity debiasing using the $\alpha(u)$ score. Users who interact with less popular items frequently receive stronger debiasing weights. Temporal trends are disabled here

2) Beta-only Configuration

```
use_alpha: false
use_beta: true
```

The configuration just enables only temporal trend debiasing using the $\beta(u)$ score. Users with strong preference for trending items receive high exposure correction.

b) Window size sensitivity Analysis Configurations

1) Window Size W = 7

```
trend_window_days: 7
```

This configuration uses a short 7-day trend window, making the model highly responsive to recent popularity changes and emerging trends.

2) Window Size W = 14

```
trend_window_days: 14
```

This configuration uses a moderate 14-day trend window, balancing responsiveness to recent trends with more stable temporal exposure estimation.

3) Window Size W = 60

```
trend_window_days: 60
```

This configuration uses a longer 60-day trend window, capturing broader long-term popularity patterns while reducing sensitivity to short-term fluctuations.

V. Experimental Results

To evaluate the proposed framework, we report both ranking accuracy metrics and fairness-diversity metrics.

Accuracy Metrics: We use Recall@10 (whether the ground-truth item appears in the top-10 recommendations) and NDCG@10 (which additionally rewards placing the correct item at higher ranks). These are evaluated per user and averaged across all test users.

Fairness and Diversity Metrics: Following Abdollahpouri et al. [1], we compute ΔGAP (change in Group Average Popularity) per user group to measure whether our method reduces the unfair impact of popularity bias on Niche users. A ΔGAP closer to zero indicates fairer recommendations. We also report: Average Popularity of recommended items (lower is better indicating less popularity bias), Coverage (number of unique items recommended across all users, higher is better indicating more variety), and Long-Tail Ratio (fraction of recommendations from the bottom 80% of items by popularity, higher is better indicating niche items getting exposure).

A. Main Comparison Results

Table 2: Main Comparision Results Summary

Method	Recall@10	NDCG@10	ΔGAP Niche	ΔGAP Diverse
Popularity Baseline	0.041432	0.019306	0.002897	0.002373
GRU4Rec CE	0.125456	0.065080	0.000792	0.000433
GRU4Rec IPS	0.122141	0.064079	0.000842	0.000449
GRU4Rec Personalized	0.123136	0.063780	0.000832	0.000441
GRU4Rec Personalized (Learned)	0.127279	0.064911	0.000823	0.000436

Method	ΔGAP Blockbuster	Avg Popularity	Coverage	Long-tail Ratio
Popularity Baseline	0.001967	0.003756	0.032550	0.000000
GRU4Rec CE	0.000671	0.001911	0.466176	0.091747
GRU4Rec IPS	0.000682	0.001934	0.463912	0.087521
GRU4Rec Personalized (Formula Based)	0.000685	0.001927	0.471271	0.092841
GRU4Rec Personalized (Learned)	0.000683	0.001922	0.465327	0.091846

The popularity baseline performs poorly across all evaluation metrics, showing that recommendation quality cannot be achieved by simply recommending popular items, very low Recall@10 and NDCG@10 values indicate the requirement of personalized sequential modelling.

Among all the sequential models, GRU4Rec trained with standard cross-entropy baseline sequential model achieves the highest-ranking accuracy because the model naturally exploits popularity patterns present in the training data. However, this comes at the cost of reduced exposure diversity and weaker long-tail recommendation behaviour.

The proposed formula-based personalized debiasing framework achieves the highest coverage and highest long-tail ratio while maintaining competitive Recall@10 and

NDCG@10 values of the baseline sequential model. Compared to standard IPS debiasing, the personalized framework has achieved better Recall@10, coverage, and long-tail exposure, demonstrating that adaptive user-specific correction is more effective than uniform debiasing in IPS.

The learned MLP-based personalized model further improves performance, achieving the best overall Recall@10 (0.1273) and NDCG@10 (0.0649) among all methods.

The comparison between the formula-based and learned MLP-based personalized debiasing models reveals a clear trade-off between ranking quality and exposure-related metrics. The learned model achieves better ranking performance, improving Recall@10 from 0.1231 to 0.1273 (~3–4% gain) and slightly increasing NDCG@10 from 0.0638 to 0.0649, indicating that end-to-end learning of debiasing weights enhances recommendation accuracy. However, the formula-based variant consistently performs better on fairness and diversity metrics, achieving higher coverage (0.4713 vs. 0.4653), slightly higher long-tail ratio (0.0928 vs. 0.0918), and marginally stronger ΔGAP improvements across user groups. This suggests that while the learned model optimizes ranking objectives more effectively, the entropy-based formulation provides more stable control over exposure distribution and item-space coverage. Overall, the results highlight a fundamental trade-off between optimizing for accuracy versus maximizing diversity and coverage, with the choice of variant depending on whether the application prioritizes ranking performance or equitable item exposure.

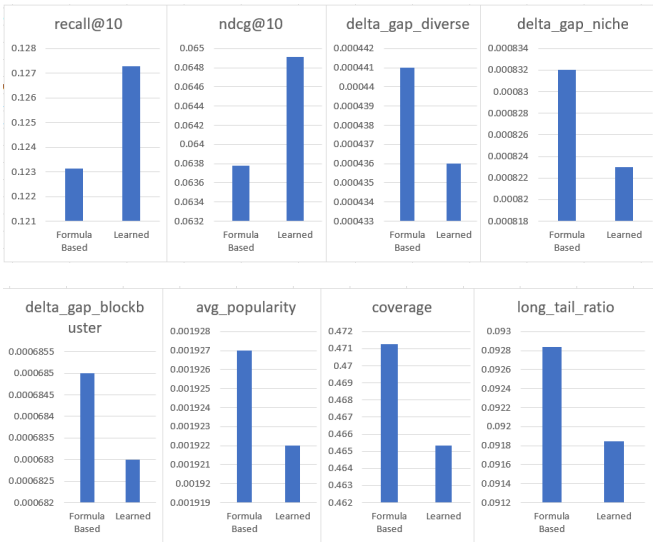

Figure 1: Comparison of Evaluation Metrics for Formula-Based and Learned Debiasing Models

B. Ablation Studies

1) Component Ablation

The ablation study evaluates the contribution of popularity debiasing and temporal trend debiasing separately. The α-only configuration achieves the highest Recall@10 and NDCG@10 values, indicating that popularity debiasing is the dominant factor influencing recommendation quality on MovieLens. This behaviour is expected because MovieLens contains strong global popularity skew where blockbuster items dominate user interactions.

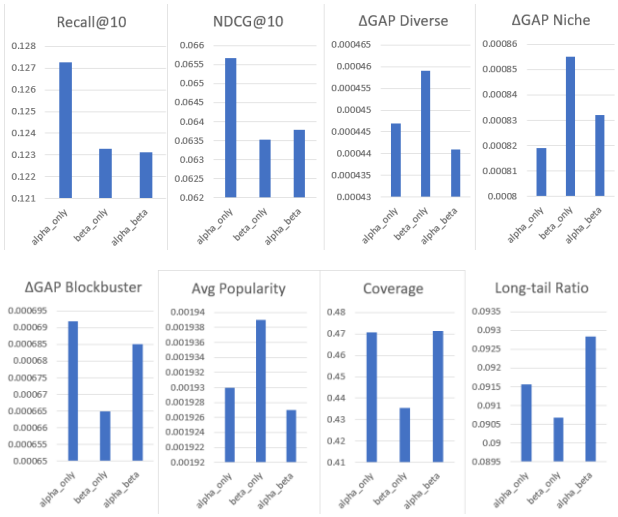

Figure 2: Comparison of Evaluation Metrics for Component Ablation Variants

The β-only configuration performs comparatively weaker in ranking metrics, suggesting that temporal trending bias is less dominant in this dataset. Nevertheless, temporal debiasing still improves fairness and diversity-related metrics, validating the usefulness of modelling trend-aware exposure.

The combined α+β configuration achieves the best balance between ranking quality, coverage, and long-tail exposure. In particular, it achieves the highest coverage and highest long-tail ratio, demonstrating that popularity-aware and trend-aware debiasing complement each other effectively.

2) Window Size Sensitivity Results

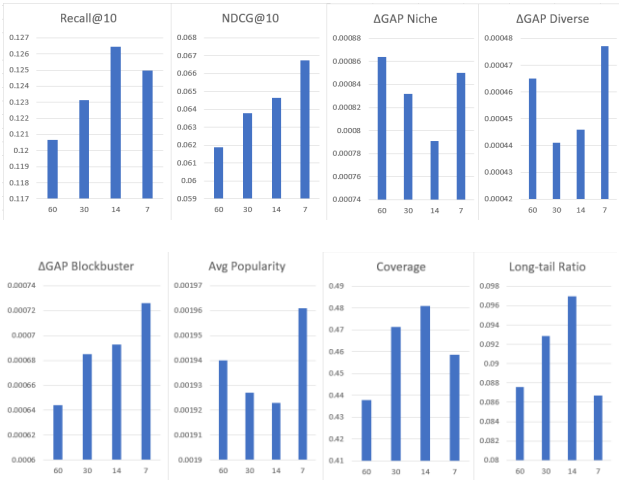

Figure 3: Comparison of Evaluation Metrics for Window Size Sensitivity

The sensitivity analysis shows that temporal window size significantly affects both ranking quality and recommendation fairness. The 14-day temporal window achieves the best overall trade-off between Recall@10, coverage, and long-tail exposure. This suggests that

moderate temporal windows capture meaningful trend dynamics while avoiding excessive short-term noise.

The 7-day configuration achieves the highest NDCG@10, indicating that very recent trends improve top-ranked recommendation precision. However, this setting reduces coverage and long-tail exposure, suggesting that short windows overemphasize rapidly trending items and short-term popularity spikes.

In contrast, the 60-day configuration performs worst across most metrics. Longer windows dilute temporal dynamics and behave similarly to static popularity estimation, reducing the model's ability to adapt to evolving user interests. These results experimentally validate the importance of localized temporal popularity modeling.

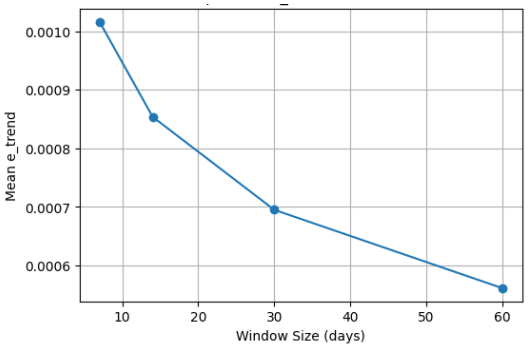

Figure 4: Trend Exposure(e_{trend}) vs Window Size

Additionally, we analyze the behavior of the underlying temporal exposure signal e_{trend} across different window sizes, which shows a consistent decrease as the window size increases. This also confirms that shorter windows produce more sensitive and reactive trend estimates, while longer windows smooth temporal variations, confirming the observed trade-off between responsiveness and stability in temporal modeling.

VI. Limitations

We acknowledge several limitations of our current approach.

A. Exposure Approximation

Our exposure estimation relies on item interaction frequency as a proxy for true exposure probability. In practice, true exposure depends on the recommendation policy deployed on the platform, which is not observable in public datasets. This is a standard limitation shared by IPS-based and counterfactual learning methods [2].

B. Dataset Scope and Generalization

Our experiments are conducted on MovieLens-1M, which exhibits relatively stable user preferences and limited temporal dynamics compared to real-world industrial settings such as social media or short-video platforms. As a result, the observed trends in popularity and temporal bias may not fully reflect highly dynamic production environments.

C. Metadata Dependency

The current implementation of the explorer-aware component relies on genre metadata to compute entropy-based user profiles. While we have discussed embedding-based clustering as an alternative, the present formulation is still dependent on structured metadata, which may not be available in all recommendation domains.

D. Offline Evaluation

All evaluations are conducted in an offline setting using standard ranking and distributional metrics. Although these metrics provide useful signals on accuracy and fairness trade-offs, they cannot fully capture user satisfaction, engagement, or long-term effects. Online A/B testing would be required to validate real-world effectiveness.

VII. Conclusion

In this work, we proposed a Personalized Debiasing framework for sequential recommendation systems that incorporates explorer-aware popularity correction, trend-aware temporal debiasing, and personalized exposure-aware loss weighting without modifying the underlying model architecture. We also propose a learned variant where a lightweight MLP jointly learns debiasing weights in an end-to-end manner.

Experiments on MovieLens-1M demonstrate that personalized debiasing consistently improves coverage and long-tail exposure while maintaining competitive ranking performance compared to standard GRU4Rec-based baseline models. The results further indicated that uniform IPS-based debiasing is less effective in capturing heterogeneous user behaviour patterns, highlighting the importance of personalization in bias mitigation. The learned variant further improves ranking performance in terms of Recall@10 and NDCG@10 while maintaining comparable diversity and fairness metrics compared to the formula-based approach. Overall, this work shows that recommendation fairness and diversity can be improved through user-aware debiasing strategies without significantly compromising ranking quality.

References

[1] H. Abdollahpouri, M. Mansoury, R. Burke, and B. Mobasher, "The Unfairness of Popularity Bias in Recommendation," in RMSE Workshop at RecSys, 2019.

[2] T. Schnabel, A. Swaminathan, A. Singh, N. Chandak, and T. Joachims, "Recommendations as Treatments: Debiasing Learning and Evaluation," in ICML, pp. 1670–1679, 2016.

[3] Ning, R. Cheng, X. Yan, B. Kao, N. Huo, N. A. H. Haldar, and B. Tang, "Debiasing Recommendation with Personal Popularity," in Proc. of the ACM Web Conference (WWW), pp. 3400–3409, 2024.

[4] J. Yang, Y. Ding, Y. Wang, P. Ren, Z. Chen, F. Cai, J. Ma, R. Zhang, Z. Ren, and X. Xin, "Debiasing Sequential Recommenders through Distributionally Robust Optimization over System Exposure," in WSDM, 2024.

[5] Y. Zhang, F. Feng, X. He, T. Wei, C. Song, G. Ling, and Y. Zhang, "Causal Intervention for Leveraging Popularity Bias in Recommendation," in SIGIR, pp. 11–20, 2021.

[6] S. Park, J. Choi, and U. Kang, "Temporal Linear Item-Item Model for Sequential Recommendation (TALE)," arXiv:2412.07382, 2024.

[7] B. Hidasi, A. Karatzoglou, L. Baltrunas, and D. Tikk, "Session-based Recommendations with Recurrent Neural Networks (GRU4Rec)," in ICLR, 2016.

[8] W. Kang and J. McAuley, "Self-Attentive Sequential Recommendation (SASRec)," in ICDM, 2018.

[9] Y. Wang, X. Zhang, Z. Liu, Z. Dong, X. Feng, R. Tang, and X. He, "Personalized Re-ranking for Improving Diversity in Live Recommender Systems," arXiv:2004.06390, 2020.

[10] O. Barkan and N. Koenigstein, "Item2Vec: Neural Item Embedding for Collaborative Filtering," in IEEE MLSP, 2016.